

Homework Set 3: High-Level Synthesis

In this homework, you are required to optimize 5 High-Level Synthesis (HLS) designs. The main goal of the optimization is to shrink the number of cycles to finish the task defined in the hardware, without changing the given clock speed. You can apply any optimizations (unroll, pipeline, memory partition, false dependency hint, and modifying the code), as long as the function of the design is the same as before. You are also required to explain your optimizations, and discuss their positive points and potentially their negative ones.

For each of the provided kernels, there is a set of tricks one can play to improve their performance. The goal is not to get the absolute best performance, but rather to identify this set of tricks and to apply them appropriately.

This homework should be done individually.

1 Environment

Assume the following fixed latencies:

Cycles per Operation	Memory Load/Store	Addition	Multiplication
Integer	1	1	4
Float	1	3	5

Similar to the lab, assume every array maps to a single BRAM, meaning each array has a single memory port. Address calculations (e.g., $2*i$ and $2*i+1$), as well as any operations occurring before & after the loop (such as variable initialization), have a negligible cost and can be ignored in the questions.

You will need to explore the Vitis HLS documentation (2025.2) for further optimizations. Since this homework doesn't use Vitis HLS, you need to specify every parameter of every pragma, including optional parameters.

2 Designs

2.1 Static HLS

```

1 void kernel1 (int array[ARRAY_SIZE]) {
2     for(int i = 0; i < ARRAY_SIZE; i++)
3         array[i] = array[i] * 5;
4 }
```

```

1 void kernel2 (int array[ARRAY_SIZE]) {
2     for(int i = 3; i < ARRAY_SIZE; i++)
3         array[i] = array[i - 1] + array[i - 2] * array[i - 3];
4 }
```

```

1 void kernel3 (float hist[ARRAY_SIZE], float weight[ARRAY_SIZE],
2               int index[ARRAY_SIZE]) {
3     for (int i = 0; i < ARRAY_SIZE; ++i)
4         hist[index[i]] = hist[index[i]] + weight[i];
5 }
```

```
1 void kernel4 (int array[ARRAY_SIZE], int index[ARRAY_SIZE], int offset) {
2     for (int i = offset + 1; i < ARRAY_SIZE - 1; ++i)
3         array[offset] = array[offset] - index[i] * array[i]
4                               + index[i] * array[i + 1];
5 }
```

```
1 float kernel5 (float bound, float a[ARRAY_SIZE], float b[ARRAY_SIZE]) {
2     int i = 0;
3     float sum = 0;
4
5     while (sum < bound && i < ARRAY_SIZE) {
6         sum = a[i] + b[i];
7         i++;
8     }
9
10    return sum;
11 }
```

2.2 Dynamic HLS

Kernel 5 contains an exit condition that depends on the runtime data. For this kernel, explain how a Dynamic HLS implementation could improve the average performance. Describe an example of data distribution where this implementation would perform significantly better than the Static HLS one, and why.

3 Submission Requirement

For each design of Section 2.1 you optimize, you are required to submit the following:

- The modified code, written clearly in your submission.
- The scheduling dataflow graph of the first three iterations of the loop, in a similar format as in the lab. The x-axis must correspond to the cycle count, while the y-axis corresponds to the iteration count. Both axis must start at 0.
- The initialization interval (II), and a brief explanation why the interval is not one, if applicable.
- The total number of cycles required to finish one iteration and also the entire task.
- The total number of resources required by your design. List the number of adders, multipliers and ports required by each array.
- A short explanation of each optimization you applied, and why. If they come with downsides, also discuss them.

The homework should be solved on paper (a typed PDF document is also accepted). You do **not** need to build or submit a Vitis HLS project. For designs that you think no optimization can be applied, you don't have to submit the modified code, but instead a reason why no further optimization is possible.